

机考模拟试题第二套

1

请你设计并实现一个满足 LRU (最近最少使用) 缓存 约束的数据结构。

实现 LRUCache 类:

LRUCache(int capacity) 以 正整数 作为容量 capacity 初始化 LRU 缓存

int get(int key) 如果关键字 key 存在于缓存中, 则返回关键字的值, 否则返回 -1 。

void put(int key, int value) 如果关键字 key 已经存在, 则变更其数据值 value ; 如果不存在, 则向缓存中插入该组 key-value 。如果插入操作导致关键字数量超过 capacity , 则应该 逐出 最久未使用的关键字。

函数 get 和 put 必须以 $O(1)$ 的平均时间复杂度运行。

示例:

```
1  输入
2  ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
3  [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
4
5  输出
6  [null, null, null, 1, null, -1, null, -1, 3, 4]
7
8  解释
9  LRUCache lRUCache = new LRUCache(2);
10 lRUCache.put(1, 1); // 缓存是 {1=1}
11 lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}
12 lRUCache.get(1);    // 返回 1
13 lRUCache.put(3, 3); // 该操作会使得关键字 2 作废, 缓存是 {1=1, 3=3}
14 lRUCache.get(2);    // 返回 -1 (未找到)
15 lRUCache.put(4, 4); // 该操作会使得关键字 1 作废, 缓存是 {4=4, 3=3}
16 lRUCache.get(1);    // 返回 -1 (未找到)
17 lRUCache.get(3);    // 返回 3
18 lRUCache.get(4);    // 返回 4
```

2

你是一个专业的小偷, 计划偷窃沿街的房屋。每间房内都藏有一定的现金, 影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统, **如果两间相邻的房屋在同一晚上被小偷闯入, 系统会自动报警。**

给定一个代表每个房屋存放金额的非负整数数组, 计算你 **不触动警报装置的情况下**, 一夜之内能够偷窃到的最高金额。

示例 1:

```
1 输入: [1,2,3,1]
2 输出: 4
3 解释: 偷窃 1 号房屋 (金额 = 1) , 然后偷窃 3 号房屋 (金额 = 3)。
4      偷窃到的最高金额 = 1 + 3 = 4 。
```

示例 2:

```
1 输入: [2,7,9,3,1]
2 输出: 12
3 解释: 偷窃 1 号房屋 (金额 = 2), 偷窃 3 号房屋 (金额 = 9), 接着偷窃 5 号房屋 (金额 =
4      1)。
      偷窃到的最高金额 = 2 + 9 + 1 = 12 。
```

提示:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 400`

3

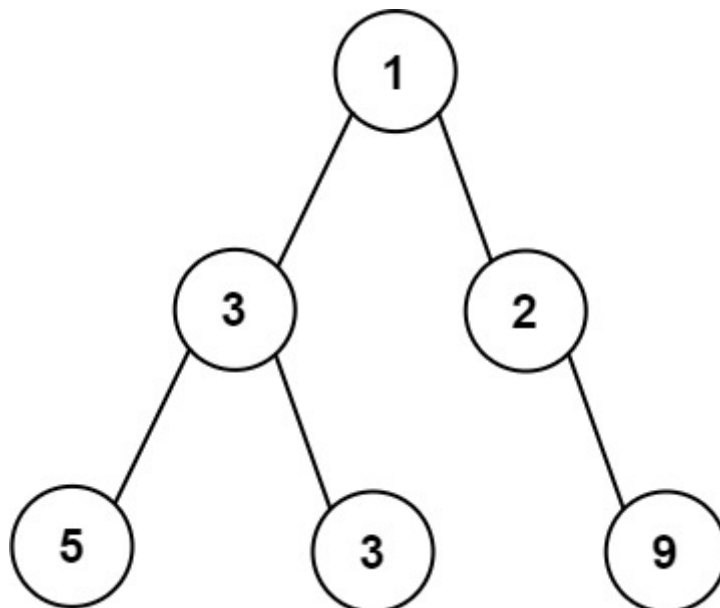
给你一棵二叉树的根节点 `root` , 返回树的 最大宽度 。

树的 最大宽度 是所有层中最大的 宽度 。

每一层的 宽度 被定义为该层最左和最右的非空节点 (即, 两个端点) 之间的长度。将这个二叉树视作与满二叉树结构相同, 两端点间会出现一些延伸到这一层的 `null` 节点, 这些 `null` 节点也计入长度。

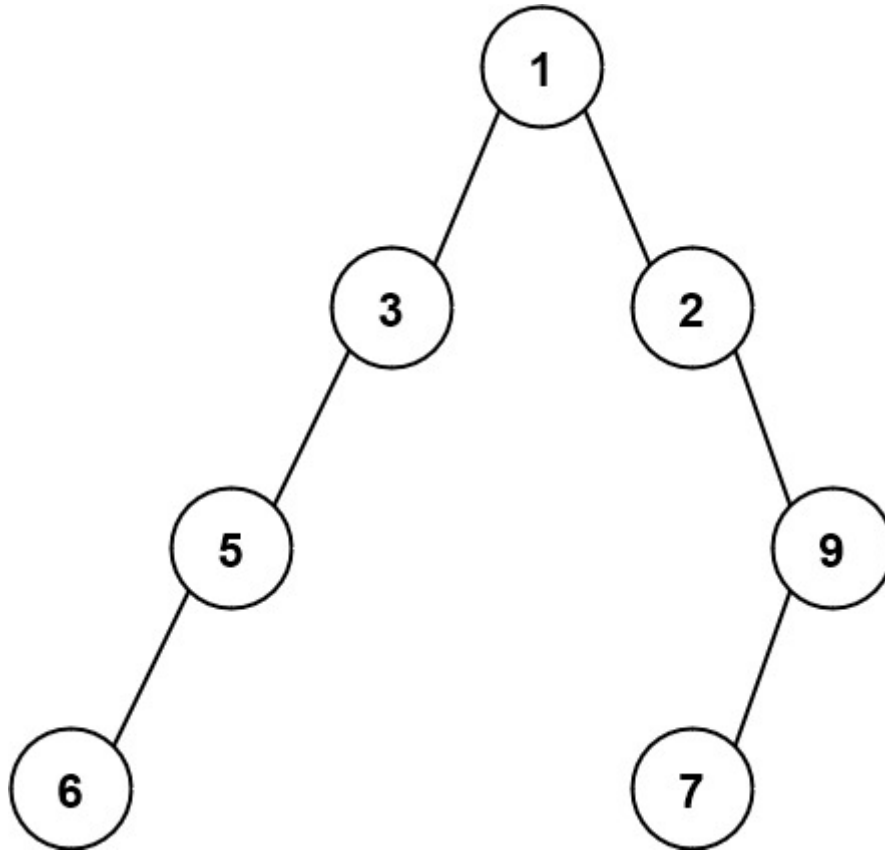
题目数据保证答案将会在 32 位 带符号整数范围内。

示例 1:



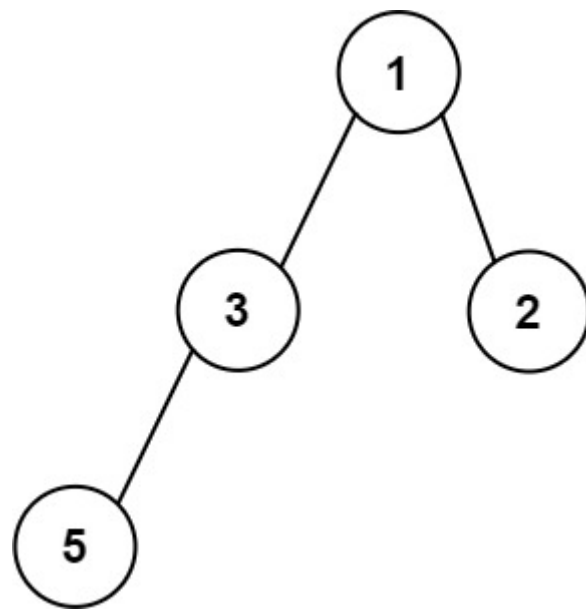
- 1 输入: root = [1,3,2,5,3,null,9]
- 2 输出: 4
- 3 解释: 最大宽度出现在树的第 3 层, 宽度为 4 (5,3,null,9) 。

示例 2:



- 1 输入: root = [1,3,2,5,null,null,9,6,null,7]
- 2 输出: 7
- 3 解释: 最大宽度出现在树的第 4 层, 宽度为 7 (6,null,null,null,null,null,7) 。

示例 3:



- 1 输入: `root = [1,3,2,5]`
- 2 输出: 2
- 3 解释: 最大宽度出现在树的第 2 层, 宽度为 2 (3,2) 。